

Comprehensive Documentation of DUL (DOLCE+DnS Ultralite) v3.34

Table of Contents

1. [Introduction & Philosophy](#)
 2. [The Cognitive-Epistemological Mindset](#)
 3. [The World Model: Fundamental Categories](#)
 4. [Context Modeling Framework](#)
 5. [Key Design Patterns](#)
 6. [Class Hierarchy Reference](#)
 7. [Property Reference](#)
 8. [Usage Guidelines](#)
-

I. Introduction & Philosophy

1.1 What is DUL?

DUL (DOLCE+DnS Ultralite) is a **foundational ontology** designed to provide reusable, domain-independent patterns for modeling any domain of discourse. It represents a **simplification and practical adaptation** of DOLCE (Descriptive Ontology for Linguistic and Cognitive Engineering), one of the most influential foundational ontologies in knowledge representation.

Key characteristics: - **Pattern-based architecture:** DUL is not just a taxonomy, but a collection of content ontology design patterns (ODPs) - **Lightweight yet foundational:** Easier to apply than full DOLCE while maintaining philosophical rigor - **Domain-agnostic:** Designed to work across physical, social, and mental domains - **Epistemologically committed:** Recognizes that our knowledge is framed by descriptions and contexts

1.2 Origins and Evolution

DUL emerged from the **Ontology Design Patterns** (ODP) initiative, which recognized that ontology engineering benefits from reusable conceptual patterns, similar to design patterns in software engineering.

From DOLCE to DUL: - **DOLCE Lite-Plus:** The parent ontology, grounded in formal ontology and philosophical analysis - **Simplification goals:** Make names more intuitive, relax some formal constraints, focus on practical applicability - **Integration of D&S:** Incorporates the Descriptions and Situations (D&S) pattern as a core architectural principle

Version 3.34 highlights: - Enhanced mereological (part-whole) support with multiple parthood relations - N-ary relation reification through Situations - Improved handling of time-indexed relations - Refinement of Person-related classes - Top-level repositioning of Situation class for broader applicability

1.3 Core Ontological Commitments

DUL makes several fundamental commitments about reality and how we model it:

1.3.1 *Descriptive vs. Prescriptive*

DUL is **descriptive**: it aims to model how people actually conceptualize reality, not to prescribe a single “correct” ontology. This philosophical stance acknowledges **ontological pluralism**—different communities and contexts may legitimately organize reality differently.

1.3.2 *The Frame-Based View*

Reality is always understood **through conceptual frames** (Descriptions). We never access “raw reality” but always reality-as-interpreted. This echoes: - **Kant’s epistemology**: We know phenomena (appearances) through mental categories, not noumena (things-in-themselves) - **Frame semantics** (Fillmore): Concepts are understood within structured mental frames - **Constructivism**: Social reality is constructed through shared conceptualizations

1.3.3 *Multiple Perspectives*

The same entity can be validly viewed from multiple perspectives: - An event can be seen as an **accomplishment**, **achievement**, **process**, or **state transition** - A physical object can be a **designed artifact**, a **biological object**, or a **refunctionalized entity** - These are not different identities but different **Situations** that frame the same entity according to different **Descriptions**

1.3.4 *Reification Strategy*

DUL heavily uses **reification**—turning relations and attributes into first-class entities: - **Qualities**: Instead of direct attributes, DUL reifies qualities (e.g., “the yellowness of Dmitri’s skin”) - **Regions**: Values are abstract regions in dimensional spaces (e.g., specific colors in color space) - **Situations**: Relations are reified as situations (e.g., “John being a student in 2024” is a situation)

This enables: - **Time-indexing**: “John was a student in 2020 but is now a teacher” - **Context-sensitivity**: “This object is a cradle in the baby room but a flower pot in the garden” - **N-ary relations**: Relations with more than two participants

1.4 Design Principles

1.4.1 *Pattern Reusability*

DUL classes and properties are designed to be **reused across domains**. Rather than creating domain-specific concepts from scratch, ontology designers: 1. Identify relevant DUL patterns 2. Specialize DUL classes for their domain 3. Combine patterns to create rich domain models

1.4.2 Simplicity over Formal Completeness

Unlike DOLCE, DUL favors: - **Intuitive naming**: “Object” instead of “Endurant”, “Event” instead of “Perdurant” - **Relaxed constraints**: Fewer formal restrictions for easier adoption - **OWL2 expressivity**: Uses OWL2 features but avoids overly complex axioms

1.4.3 Social and Cognitive Orientation

DUL gives special prominence to: - **Social objects**: Entities that exist through communication and shared understanding - **Information**: Abstract information vs. concrete realizations - **Descriptions**: Conceptual schemas that organize experience - **Agency**: Agents, roles, tasks, plans, and intentional action

II. The Cognitive-Epistemological Mindset

2.1 The Central Insight: Descriptions and Situations

The most distinctive feature of DUL is the **D&S (Descriptions and Situations) pattern**, which embodies a fundamentally **epistemological** approach to ontology.

Core Distinction:

- **Description**: A conceptual schema, theory, or frame that defines concepts and their relationships
- **Situation**: A view on reality that satisfies (is consistent with) a description

Example: - **Description**: “A recipe for chocolate cake” (defines ingredients, procedures, roles like ‘baker’, tasks like ‘mixing’) - **Situation**: “My baking session this morning” (satisfies the recipe: I played the baker role, I executed the mixing task, etc.)

This separation allows us to model: 1. **The same reality under different theories**: An avalanche can be framed as a natural process (physics) or as a crime scene (law) 2. **Theories without instances**: We can model the recipe even if no one has ever baked that cake 3. **Multiple interpretations**: The same physical event can satisfy different descriptions (diagnosis, narrative, legal norm)

2.2 The Epistemological Commitment

DUL recognizes that **knowledge is always mediated by conceptual frameworks**. This has profound implications:

2.2.1 No Direct Access to Reality

We never model “raw objects” or “raw events”—we model: - Objects-as-classified (a physical entity as a “car”, as a “weapon”, as “evidence”) - Events-as-interpreted (a physical process as an “action”, as an “accident”, as a “crime”)

DUL makes this explicit through: - **classifies/isClassifiedBy**: Concepts classify entities

within contexts - **satisfies/isSatisfiedBy**: Situations satisfy descriptions that provide interpretation

2.2.2 Context Dependence

The “same” entity can have different identities in different contexts: - An old cradle is a **baby furniture** in a museum (satisfying a historical design description) - The same cradle is a **flower pot** in a garden (satisfying a home decoration description)

Both are valid, non-contradictory views—they are different Situations that include the same physical object.

2.2.3 Observer Relativity

DUL acknowledges that **observers create contexts**: - A Situation is “a view created by an observer on the basis of a frame” - Different observers (or the same observer at different times) can create different situations from the same data

2.3 The Social Construction of Reality

DUL distinguishes **PhysicalObject** and **SocialObject** as disjoint:

PhysicalObject

- Has spatial location and (typically) mass
- Exists independently of human conceptualization
- Examples: rocks, trees, human bodies, buildings

SocialObject

- Exists **only through communication** and shared understanding
- Must be **expressed by InformationObjects** (speech, writing, gestures)
- Examples: laws, roles, organizations, concepts, money, marriages

Key insight: Social reality is **ontologically dependent** on information: - A marriage exists because it is expressed in legal documents and social practices - A role (e.g., “professor”) exists because it is defined in institutional descriptions - An organization exists because its structure is documented and communicated

This captures the **constitutive role of language and communication** in creating social facts (echoing John Searle’s philosophy of social ontology).

2.4 Information and Meaning

DUL provides a sophisticated model of **information** that distinguishes:

InformationObject (Abstract)

The information content itself, independent of physical realization: - The 3rd Gymnopedie by Satie (the musical composition) - The text of the Italian Constitution (the legal content) - The concept “dog” (the mental/social construct)

InformationRealization (Concrete)

Physical or event-based realizations: - A printed music sheet, a piano performance, a recording - A specific book copy, an oral recitation - Utterances of the word “dog”, written tokens

Relation: realizes connects InformationRealization to InformationObject

expresses Relation

InformationObjects **express** SocialObjects (their “meanings”): - The term “professor” expresses the Role concept ‘Professor’ - The recipe text expresses the Plan for making cake - The Constitution text expresses the legal Norm system

This three-level model (Physical Realization → Abstract Information → Social Meaning) provides a foundation for: - Semantic web and knowledge representation - Cultural heritage documentation - Legal and institutional modeling - Discourse and narrative analysis

2.5 The Cognitive Premise: Concepts and Classification

DUL models the **cognitive act of categorization**:

Concept

A SocialObject that **classifies** entities: - Defined within a Description - Can be reused across descriptions - Acts as an intensional category (the “idea” of something)

Classification (Time-Indexed)

A special Situation that captures: - What concept classifies what entity - At what time interval - Within what larger context

Example: “My old cradle **is classified as** a flower pot **in June 2024** (within the situation of my garden decoration project)”

This allows: - **Dynamic classification:** Entities can be reclassified over time - **Context-dependent classification:** Different classifications in different situations - **Multiple simultaneous classifications:** The cradle is still a cradle (historically) even while functioning as a flower pot

2.6 The Quality-Region Pattern

DUL reifies attributes through the **Quality-Region** pattern:

Quality

An individual **aspect** of an entity: - Cannot exist without the entity (dependent) - The specific yellowness of Dmitri’s skin - The specific height of the Eiffel Tower

Region

An abstract value in a dimensional space: - Independent of particular entities - “180 cm” in the space of possible heights - “Yellow” in color space

Relation: hasRegion connects Quality to Region (the quality’s value)

Why this complexity? 1. **Observation vs. reality:** The same quality (Dmitri’s skin color) can be measured differently (RGB values, wavelengths, color names) 2. **Temporal change:** Qualities persist even as their regions change (Dmitri’s skin can tan) 3. **Parameter-based constraints:** Concepts can constrain regions (Role ‘Driver’ requires Parameter ‘MinimumAge’ > 16)

2.7 Epistemological vs. Ontological Modeling

DUL allows modeling at two levels:

Ontological Level

What exists in the domain: - Physical objects, events, their parts and qualities - Mereological (part-whole) relations - Spatial and temporal location

Epistemological Level

How we conceptualize and organize what exists: - Descriptions, concepts, roles, tasks - Situations, classifications, contexts - Information objects expressing knowledge

Most ontologies focus only on the ontological level. DUL’s innovation is **integrating both**, acknowledging that: - Domain models are not “reality” but “conceptualizations of reality” - Different conceptualizations can coexist (pluralism) - The act of modeling itself involves descriptions and situations

III. The World Model: Fundamental Categories

3.1 The Top-Level Ontology

DUL’s root is **Entity**, defined as: “*Anything: real, possible, or imaginary, which some modeller wants to talk about for some purpose.*”

This maximally permissive definition reflects DUL’s **pluralistic** stance: if someone needs to model something, it qualifies as an Entity.

Direct subclasses of Entity:

Entity	
— Abstract	(not located in space-time)
— Event	(temporal extent, participants)
— InformationEntity	(information: abstract or concrete)
— Object	(spatial location)

└─ Quality	(aspects of entities)
└─ Situation	(contextualized views)

3.2 Abstract Entities

Abstract: Entities not located in space-time.

Key subclasses:

FormalEntity - Formally defined, context-independent, “Platonic” entities - Mathematical entities: sets, categories, functions - Distinguished from Concepts (which are social/context-dependent) - Example: The mathematical set $\mathbb{N}$ (natural numbers)

Region - Values in dimensional spaces - Subclasses represent different dimensions: - **SpaceRegion**: Spatial coordinates, geometries - **TimeInterval**: Temporal extents - **Amount**: Quantities (mass, volume, count) - **PhysicalAttribute**: Physical measurements (temperature, pressure) - **SocialObjectAttribute**: Social attributes (salary level, legal status)

Why Regions are Abstract: - The number “42” exists independently of any particular 42 objects - The color “red” exists independently of any particular red object - They inhabit abstract dimensional spaces, not physical space-time

3.3 Objects

Object: Entities with spatial location, participating in events.

The Physical vs. Social Divide

DUL makes **PhysicalObject** and **SocialObject** disjoint—a fundamental ontological distinction:

PhysicalObject

- Has spatial region
- Has (typically) mass
- Exists independently of communication

Subclasses: - **PhysicalBody**: Natural material objects - **BiologicalObject**: Living organisms - **ChemicalObject**: Chemical substances - **Substance**: Materials (water, steel, DNA) - **PhysicalArtifact**: Human-made physical objects - **DesignedArtifact**: Artifacts with explicit design (cars, buildings) - **PhysicalPlace**: Locations understood as physical regions

SocialObject

- Exists through communication (in “some communication Event”)
- Must be **expressed by InformationObject**
- Disjoint from PhysicalObject

Subclasses: - **Description**: Conceptual schemas (theories, frames) - Plan, Design, Diagnosis, Norm, Contract, Goal, Theory, Narrative - **Concept**: Categories defined in descriptions - Role, Task, EventType, Parameter - **Collection**: Containers for entities sharing properties -

Configuration, Collective, TypeCollection - **InformationObject**: Abstract information pieces
- **Place**: Socially constructed locations (countries, neighborhoods)

Agent (Cross-cutting)

A special Object category for entities with agency:

- **PhysicalAgent**: Biological agents (organisms, persons as physical beings)
- **SocialAgent**: Socially constructed agents
 - **Organization**: Structured institutions (companies, governments)
 - **CollectiveAgent**: Groups acting collectively
 - **Group**: Coordinated collectives (committees, teams)
 - **Community**: Large-scale collectives (societies, movements)

Person is modeled with two facets: - **NaturalPerson**: The physical/biological aspect (extends Person and PhysicalAgent) - **SocialPerson**: The social/legal aspect (extends Person)

This dual modeling reflects that a person is both a physical organism and a social entity with roles, rights, and legal status.

3.4 Events

Event: “Any physical, social, or mental process, event, or state.”

DUL’s Aspectual Neutrality

The Event class documentation provides extensive philosophical discussion:

The Problem: The same real-world occurrence can be viewed as: - An **accomplishment** (process leading to a result) - An **achievement** (the result state) - A **punctual event** (time-collapsed) - A **transition** (change between states)

DUL’s Solution: Don’t classify Events by aspect—use **Situations** for aspectual views: - The Event “rock erosion in Sinni valley” has a single identity - **ErosionAsAccomplishment**: A Situation viewing it as a process - **ErosionAsTransition**: A Situation viewing it as a state change - Both Situations include the same Event but satisfy different Descriptions (theories of aspect)

Subclasses:

Action - Event with at least one **Agent** participant - Executes a **Task** typically defined in a Plan - Intentional, goal-directed

Process - Event without agentive focus - Natural or social processes (erosion, inflation, aging)

State (implied but not always distinguished) - Stative events (being tall, being red)

3.5 Qualities

Quality: Individual aspects of entities that cannot exist independently.

Examples: - The specific yellowness of Dmitri's skin (not yellowness in general) - The specific height of this building (not 180cm in general) - The specific beauty of this painting

Key relations: - `isQualityOf`: Links quality to its bearer (entity) - `hasRegion`: Links quality to its value (region)

When to use Qualities: DUL advises using Qualities only when **individual aspects matter**:

- **Relevant:** Antique furniture appraisal (each piece's individual patina, color, texture) -

Irrelevant: Assembly line quality control (only conformance to design parameters matters)

For most domains, direct attributes suffice. Qualities enable: - Fine-grained observation modeling - Temporal change tracking (same quality, different regions over time) - Multi-perspective measurement (same quality, different parameters)

3.6 Situations

Situation: "A view, consistent with a Description, on a set of entities."

Dual Nature:

1. **Epistemological:** A framed interpretation of reality
 - Created by observers applying conceptual frames
 - Multiple Situations can include the same entities (different framings)
2. **Technical:** Reified n-ary relations
 - Binary relations project from Situations via `isSettingFor`
 - Enables time-indexing and parameter-based relations

Key Subclasses:

TimeIndexedRelation - Situations specifically for temporal context - **Classification:** Time-indexed concept-entity classification - **Parthood:** Time-indexed part-whole relations

PlanExecution - Situation of executing a Plan - Links Actions to Tasks, Agents to Roles

WorkflowExecution - Situation of executing a Workflow - Temporal sequencing of tasks

Transition - Situation of change between states

3.7 InformationEntity

InformationEntity: A catchall for information, abstract or concrete.

Motivation: Bypass ambiguities in ordinary language: - "The 3rd Gymnopedie" could mean the composition (abstract) or a particular recording (concrete) - **InformationEntity** covers both, allowing underspecification when convenient

Subclasses: - **InformationObject** (Abstract): The information content - **InformationRealization** (Concrete): Physical/event realization

Relation: `realizes` connects realization to object

3.8 ObjectAggregate

ObjectAggregate: Aggregates of distributed objects from a Collection.

Distinction: - **Collection:** First-order entity (a social object unifying members conceptually)
- **ObjectAggregate:** The distributed physical aggregate of members - **Set:** Second-order formal entity (abstract set in mathematical sense)

Example: - **Collection:** “The Louvre Egyptian collection” (institutional concept) -
ObjectAggregate: The physical artifacts distributed in display cases - **Set:** The mathematical set {artifact₁, artifact₂, ...}

IV. Context Modeling Framework

4.1 What is Context in DUL?

DUL provides rich machinery for **context-sensitive modeling**—representing how facts and relations depend on: - **Temporal context:** When does this relation hold? - **Conceptual context:** Under what theory/frame is this interpreted? - **Social context:** Within what institution/agreement is this valid?

The primary mechanism is the **Situation** class and related patterns.

4.2 The Situation Pattern

Core Structure:

```
Situation --satisfies--> Description
          --isSettingFor--> Entity (multiple)
```

A Situation: - Includes multiple entities (via `isSettingFor` and subproperties) - Satisfies a Description (the conceptual frame) - Has a coherent identity as a contextualized view

Example: Coffee Preparation

Situation: “My coffee preparation this morning” - **satisfies:** Recipe Description (defines roles, tasks, ingredients) - **includesAgent:** Me (as baker role) - **includesAction:** Grinding action, brewing action - **includesObject:** Coffee beans, water, coffee maker - **includesTime:** This morning (8:00-8:15 AM)

The Situation **unifies** these entities into a coherent context, interpreted through the Recipe frame.

4.3 Descriptions: Conceptual Contexts

Description is the most important SocialObject subclass—it represents **conceptualizations**.

Types of Descriptions:

Plan - Describes actions to be executed - Defines roles (who), tasks (what), parameters (how)
- Example: Project plan, recipe, travel itinerary - **Specializations:** Project, Workflow

Design - Describes structure and function - Provides rationale for entity construction -
Example: Architectural design, software design, mechanism design

Diagnosis - Describes system state for control or explanation - Example: Medical diagnosis, fault diagnosis, situation assessment

Norm - Describes social rules and obligations - Example: Traffic law, ethical code, institutional policy

Contract - Describes agreements between parties - Defines roles (parties) and tasks (obligations) - Example: Employment contract, treaty, license

Goal - Describes desired situation - Typically associated with Plan for achievement -
Example: Business objective, personal aspiration

Theory - Describes general conceptual framework - Example: Scientific theory, philosophical system

Narrative - Describes event sequences and their interpretation - Example: Historical account, story, case study

The defines Relation

Descriptions **define** Concepts:

Description --defines--> Concept
Concept --isDefinedIn--> Description

Example: - Recipe (Description) defines “Ingredient” (Role), “Mixing” (Task) - Traffic Law (Norm) defines “Driver” (Role), “Vehicle” (Concept)

The satisfies Relation

Situations **satisfy** Descriptions:

Situation --satisfies--> Description
Description --isSatisfiedBy--> Situation

Example: - My coffee preparation (Situation) satisfies the recipe (Plan) - Today’s traffic (Situation) satisfies (or violates) traffic law (Norm)

4.4 Classification: Concept-Based Contexts

Classification is a special Situation subclass for **time-indexed classification**:

Classification (a Situation)
--satisfies--> Description (defines the Concept)
--isSettingFor--> Entity (the classified entity)
--isSettingFor--> Concept (the classifying concept)
--isSettingFor--> TimeInterval (when classification holds)

Why Classification as Situation?

Direct relation Concept classifies Entity is atemporal and acontextual. But: -
Classifications change over time: “John is a student (in 2020), then a teacher (in 2024)” -
Classifications depend on context: “This object is furniture (in museum), but art (in gallery)”

Solution: Reify classification as a Situation: - **Simple:** Concept 'Student' classifies Entity 'John' (timeless) - **Contextual:** Classification Situation satisfying University Description, including John, 'Student' concept, and TimeInterval '2020-2024'

4.5 Parthood: Mereological Contexts

DUL provides time-indexed parthood through the **Parthood** Situation:

Parthood (a TimeIndexedRelation)
--includesWhole--> Entity (the whole)
--includesPart--> Entity (the part)
--includesTime--> TimeInterval (when parthood holds)

Why Time-Indexed Parthood?

Parts change: - “My bike has a luggage rack (since March 29, 2021)” - “This ship had a mast (until the storm destroyed it)”

Direct hasPart relation is timeless. Parthood Situation adds temporal context.

Multiple Parthood Relations

DUL distinguishes several mereological relations:

hasPart / isPartOf - Reflexive + transitive (classical mereology) - “The body has a brain as part” - “2024 is part of the 21st century”

hasProperPart / isProperPartOf - Transitive + asymmetric (irreflexive) - Strict parthood (part ≠ whole)

hasComponent / isComponentOf - Asymmetric (not transitive) - For designed systems with structural roles - “The car has an engine as component”

hasConstituent / isConstituentOf - Cross-layer parthood - Links entities from different ontological strata - “The person has molecules as constituents” - “The social system has persons as constituents”

4.6 N-ary Relation Reification

Standard ontologies struggle with n-ary relations (more than 2 participants): - **Traditional:** “John gave Mary a book on Tuesday” — how to model?

DUL Solution: Use Situation as a reified relation:

Situation: Giving₁
--satisfies--> GivingDescription
--includesAgent--> John (giver)
--includesAgent--> Mary (receiver)

```
--includesObject--> Book1 (transferred object)
--includesTime--> Tuesday (when)
```

All binary relations project from the Situation via `isSettingFor` (and subproperties like `includesAgent`).

Advantages: - Arbitrary arity (any number of participants) - Add context freely (time, place, conditions) - Attach qualifications (degree, modality, evidentiality)

4.7 Practical Context Modeling Examples

Example 1: Role-Based Context

Scenario: “John is a professor at MIT in 2024”

Model:

```
Classification1 (Situation)
--satisfies--> AcademicStructure (Description defining 'Professor' Role)
--isSettingFor--> John (Person)
--isSettingFor--> 'Professor' (Role)
--isSettingFor--> MIT (Organization)
--isSettingFor--> TimeInterval2024
```

This captures: - What role (Professor) - Who plays it (John) - In what institution (MIT) - When (2024)

Example 2: Evolving Design Context

Scenario: “An old cradle is refunctionalized as a flower pot”

Model:

```
OriginalSituation (satisfying Baby Furniture Design)
--isSettingFor--> Cradle1
--includesTime--> HistoricalPeriod
--satisfies--> BabyFurnitureDesign (Description)
--defines--> 'Cradle' (Role)
```

```
RefunctionalizedSituation (satisfying Garden Decoration Design)
--isSettingFor--> Cradle1 (same object!)
--includesTime--> CurrentPeriod
--satisfies--> GardenDecorationDesign (Description)
--defines--> 'FlowerPot' (Role)
```

The same physical object (Cradle₁) participates in different Situations satisfying different Designs.

Example 3: Multi-Perspective Event Analysis

Scenario: “An avalanche that may have been triggered intentionally”

Model:

Event: Avalanche₁

PhysicalSituation

```
--isSettingFor--> Avalanche1
--satisfies--> PhysicsDescription (natural forces theory)
```

LegalSituation

```
--isSettingFor--> Avalanche1
--isSettingFor--> SuspectPerson
--satisfies--> CriminalLawDescription (intentional causation theory)
```

Same event, two interpretations, both valid in their respective descriptive contexts.

4.8 Situation vs. Context (Conceptual Clarification)

Situation (DUL) $\approx$ **Context** (informal usage), but more precise:

- **Context** (informal): Vague notion of “relevant circumstances”
- **Situation** (DUL): Formal reification with:
 - Identity (a first-order entity)
 - Structure (isSettingFor relations to included entities)
 - Interpretation (satisfies a Description providing meaning)

DUL’s Situations enable **context-aware reasoning**: - What entities are co-present in a context? (Follow isSettingFor) - What concepts apply in this context? (Check Description that is satisfied) - What are the temporal bounds of this context? (Check includesTime)

V. Key Design Patterns

5.1 Overview of DUL Patterns

DUL is not just a taxonomy but a **library of reusable patterns**. Each pattern solves a recurring modeling problem.

Major Patterns: 1. Descriptions & Situations (D&S) 2. Quality-Region 3. Participation 4. Information Realization 5. Mereological Patterns 6. Classification 7. Place 8. Sequence

5.2 The Descriptions & Situations (D&S) Pattern

Problem: How to model context-dependent relations and time-varying properties?

Solution: Separate **intensional** (conceptual) and **extensional** (actual) aspects:

Description (intensional)

```
--defines--> Concept, Role, Task, Parameter
```

Situation (extensional)

```
--satisfies--> Description
```

```
--isSettingFor--> Entity (classified, related, contextualized)
```

When to use: - Role assignment: Agent plays Role in Situation - Task execution: Action executes Task in Situation - Time-indexed relations: Relations hold within temporal Situations - Multi-perspective modeling: Same entities, different Situations/Descriptions

Example Pattern Instance:

```

Recipe (Plan, a Description)
  --defines--> 'Ingredient' (Role)
  --defines--> 'Mixing' (Task)

CookingSession (PlanExecution, a Situation)
  --satisfies--> Recipe
  --includesAgent--> Chef1 (plays 'Cook' Role)
  --includesAction--> Mixing1 (executes 'Mixing' Task)
  --includesObject--> Flour1 (plays 'Ingredient' Role)

```

5.3 The Quality-Region Pattern

Problem: How to model attributes with: - Multiple measurement systems - Temporal change
- Observational relativity

Solution: Reify attributes as Qualities, link to abstract Regions:

```
Entity --hasQuality--> Quality --hasRegion--> Region
```

When to use: - Scientific observation: Same phenomenon, different measurements -
Individual aspects matter: Not just “height” but “this building’s height” - Parameterized
constraints: Concepts constrain regions through parameters

Example Pattern Instance:

```

DmitrisSkin (Quality)
  --isQualityOf--> Dmitri (Person)
  --hasRegion--> Yellow1 (SocialObjectAttribute / Color value)

MeasurementSituation
  --includesObject--> Dmitri
  --includesQuality--> DmitrisSkin
  --satisfies--> ColorimetryDescription
  --defines--> RGBParameter

```

5.4 The Participation Pattern

Problem: How to link events and objects?

Solution: Symmetric participation relation:

```

Event --hasParticipant--> Object
Object --isParticipantIn--> Event

```

Extensions: - **Agent participation:** Action –hasParticipant→ Agent (agent-specific) - **Co-participation:** Object –coparticipatesWith→ Object (derived from shared event)

When to use: - Event modeling: Who/what is involved? - Provenance: What events affected this object? - Social network analysis: Who co-participated in events?

Example Pattern Instance:

```

TennisMatch1 (Event)
  --hasParticipant--> Vitas (Agent)
  --hasParticipant--> Jimmy (Agent)
  --hasParticipant--> TennisBall1 (Object)

Vitas --coparticipatesWith--> Jimmy (inferred)

```

5.5 The Information Realization Pattern

Problem: Distinguish abstract information from concrete manifestations.

Solution: Two-level information model:

```
InformationRealization --realizes--> InformationObject --expresses-->
SocialObject
```

When to use: - Cultural heritage: Work vs. manifestations (FRBR-compatible) - Legal documents: Legal content vs. physical copies - Knowledge representation: Concepts vs. terms expressing them

Example Pattern Instance:

```
Constitution_ItalianRepublic (InformationObject)
  --expresses--> ItalianLegalSystem (Description/Norm)

PhysicalCopy1 (InformationRealization / PhysicalObject)
  --realizes--> Constitution_ItalianRepublic

OralRecitation1 (InformationRealization / Event)
  --realizes--> Constitution_ItalianRepublic
```

5.6 Mereological Patterns

Problem: Model part-whole relations with different characteristics.

Solution: Multiple parthood properties:

Pattern A: Reflexive Part-Whole

```
Entity --hasPart--> Entity (reflexive, transitive)
```

Use when: General decomposition, entity is part of itself (mereologically valid)

Pattern B: Proper Part-Whole

```
Entity --hasProperPart--> Entity (asymmetric, transitive)
```

Use when: Strict decomposition, part ≠ whole

Pattern C: Component-System

```
Object --hasComponent--> Object (asymmetric, NOT transitive)
```

Use when: Designed artifacts with direct structural components

Pattern D: Constitution

```
Entity --hasConstituent--> Entity (cross-layer)
```

Use when: Different ontological strata (social/physical, organism/molecular)

Example Pattern Instance:

```
Car1 (DesignedArtifact)
  --hasComponent--> Engine1 (direct component)
  --hasComponent--> Wheel1 (direct component)

Engine1
  --hasProperPart--> Piston1 (engine part)
```


Car₁ --hasPart--> Piston₁ (inferred via transitivity of hasPart, if rules applied)

Person₁ (BiologicalObject)
--hasConstituent--> Molecule₁ (cross-layer: organism → molecular)

5.7 The Classification Pattern

Problem: Context and time-dependent classification.

Solution: Classification as a special Situation:

Classification (TimeIndexedRelation)
--satisfies--> Description (defines Concept)
--isSettingFor--> Concept (classifier)
--isSettingFor--> Entity (classified)
--isSettingFor--> TimeInterval (when)

When to use: - Role assignment: Agents play different roles at different times - Taxonomic classification: Species membership, artifact categories - Status tracking: Legal status, health status, employment status over time

Example Pattern Instance:

Classification₁
--satisfies--> TrafficLaw (Norm defining vehicle types)
--isSettingFor--> Vehicle₁ (Entity)
--isSettingFor--> 'Truck' (Concept)
--isSettingFor--> Interval₂₀₂₄ (when registered as truck)

Classification₂
--satisfies--> HistoricalArchive (Description)
--isSettingFor--> Vehicle₁ (same entity)
--isSettingFor--> 'ClassicCar' (different Concept)
--isSettingFor--> Interval₂₀₃₀ (after retirement from service)

5.8 The Place Pattern

Problem: Locations are both physical and social constructs.

Solution: Distinguish PhysicalPlace and Place:

PhysicalPlace (PhysicalObject)
--hasSpaceRegion--> SpaceRegion (geometric location)

Place (SocialObject)
--isExpressedBy--> InformationObject (social definition)
--describes--> PhysicalPlace (optional)

When to use: - Physical location: Coordinates, geometric regions - Social/administrative places: Countries, neighborhoods, institutions - Hybrid: “Paris” is both a geographic region and a social construct

Example Pattern Instance:

Paris_Geographic (PhysicalPlace)
--hasSpaceRegion--> Region48.8566N_2.3522E

Paris_City (Place, SocialObject)
--isExpressedBy--> ParisCharter (InformationObject / legal document)

```
--describes--> Paris_Geographic
--isMemberOf--> FrenchCities (Collection)
```

5.9 The Sequence Pattern

Problem: Model temporal or logical ordering.

Solution: Precedence relations:

```
Entity --precedes--> Entity (transitive)
Entity --directlyPrecedes--> Entity (intransitive, strict adjacency)
```

When to use: - Workflow: Task sequences - Narrative: Event chronology - Procedure: Step ordering

Example Pattern Instance:

```
Workflow1 (Description)
  --defines--> Task1, Task2, Task3

Task1 --directlyPrecedes--> Task2
Task2 --directlyPrecedes--> Task3

Task1 --precedes--> Task3 (inferred via transitivity)

WorkflowExecution1 (Situation)
  --satisfies--> Workflow1
  --includesAction--> Action1 (executes Task1)
  --includesAction--> Action2 (executes Task2)
  --includesAction--> Action3 (executes Task3)
```

5.10 Pattern Composition

Real-world modeling typically **combines multiple patterns**:

Example: Project Management Ontology

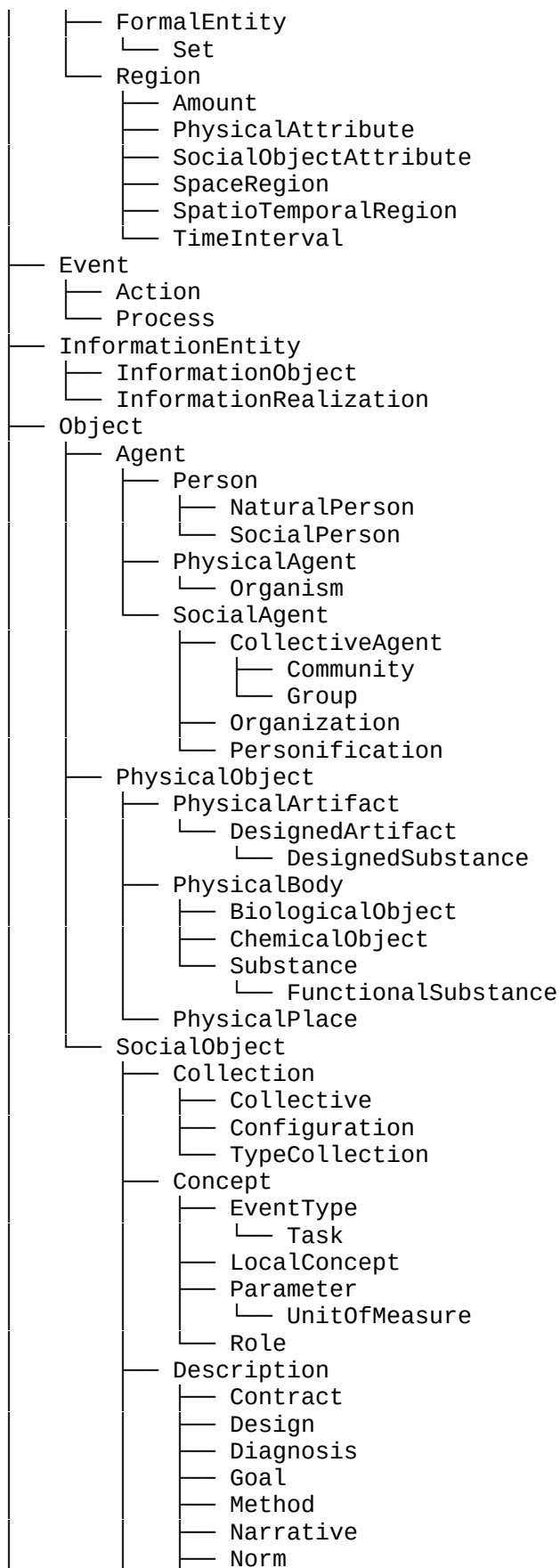
Uses: 1. **D&S**: Project (Plan) defines Roles/Tasks, ProjectExecution (Situation) satisfies Project 2. **Participation**: Actions have Agent participants 3. **Classification**: Agents classified by Roles (ProjectManager, Developer) 4. **Sequence**: Tasks ordered by precedence 5. **Quality-Region**: Project attributes (budget, timeline) as Qualities with Amount/TimeInterval Regions 6. **Mereological**: Project has sub-projects as proper parts

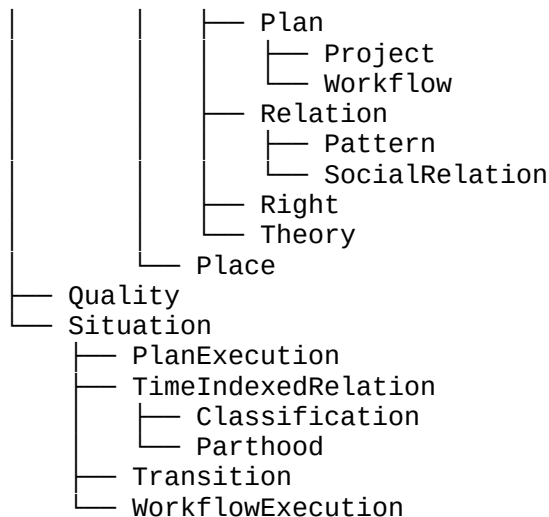
Pattern Integration Strategy: 1. Identify core entities (agents, resources, activities) 2. Apply D&S for role/task structure 3. Apply Participation for event-object links 4. Apply Classification for role assignment 5. Apply Sequence for workflow ordering 6. Apply Quality-Region for measurements/KPIs 7. Apply Mereology for hierarchical decomposition

VI. Class Hierarchy Reference

6.1 Complete Class Hierarchy

```
Entity
|— Abstract
```





ObjectAggregate (parallel hierarchy, not under Entity)

6.2 Detailed Class Descriptions

Abstract Branch

Abstract - Entities not located in space-time - Mathematical entities, formal semantics, regions

FormalEntity - Formally defined, independent of social context - “Platonic entities”: sets, categories, constants, variables - Distinguished from Concept (social) and InformationObject (localized)

Set - Mathematical sets in the extensional sense - Second-order entities (classes as sets)

Region - Values in dimensional spaces - For qualities to be measured/observed

Amount - Quantities independent of measurement - Mass, volume, count, monetary amounts

PhysicalAttribute - Physical dimensions: temperature, pressure, velocity, wavelength

SocialObjectAttribute - Social dimensions: salary level, legal status, reputation score

SpaceRegion - Spatial coordinates, geometries - Absolute locations

TimeInterval - Temporal extents - Start/end points, durations

SpatioTemporalRegion - Combined space-time regions - For events with spatial extent

Event Branch

Event - Any physical, social, or mental process, event, or state - Has temporal extent - Has object participants - Aspectually neutral (use Situations for aspectual views)

Action - Event with Agent participants - Executes Task (typically defined in Plan) - Intentional, goal-directed activity

Process - Event without agentive focus (or non-human agents) - Natural processes: erosion, combustion, growth - Social processes: inflation, cultural change

InformationEntity Branch

InformationEntity - Catchall for information (abstract or concrete) - Bypasses ambiguities in ordinary language

InformationObject - Abstract information: musical compositions, texts, concepts - SocialObject (requires social context) - Expressed by symbols, realized by physical/event entities

InformationRealization - Concrete realizations: written documents, performances, recordings - Can be PhysicalObject (book) or Event (performance) - Realizes InformationObject

Object Branch

Object - Entities with spatial location - Participate in events - Can be physical or social

Agent Subtree

Agent - Agentive objects (capable of intentional action) - Can play Roles, execute Tasks

PhysicalAgent - Biological agents: organisms, natural persons - Physical basis for agency

Organism - Biological objects with self-reproduction capability - Includes all living beings with agency

SocialAgent - Socially constructed agents - Organizations, collective agents, personifications

Organization - Internally structured, conventionally created agents - Requires specific Roles and Agents to act - Examples: companies, governments, institutions

CollectiveAgent - Agents acted by members of a Collective - Less stable than Organizations (no dedicated internal structure) - Examples: social movements, action groups

Group - CollectiveAgent with shared conceptualization of a SocialRelation - Coordinated intentionality - Examples: committees, teams, clubs

Community - Large-scale CollectiveAgent - Examples: neighborhoods, societies, cultural communities

Person - Dual-aspect modeling of human persons

NaturalPerson - Physical/biological aspect: “that person walking down the street” - Extends Person and PhysicalAgent

SocialPerson - Social/legal aspect: rights, roles, legal status - Extends Person

Personification - Non-human entities treated as agents (social attribution) - Examples: AI assistants, corporate “personalities”, deities

PhysicalObject Subtree

PhysicalObject - Proper spatial region - Typically has mass (varies by epistemological status) - Disjoint from SocialObject

PhysicalBody - Natural material objects - Examples: rocks, biological bodies, chemical samples

BiologicalObject - Objects with biological characteristics - Includes organisms and biological structures

ChemicalObject - Objects characterized by chemical composition - Examples: molecules, chemical compounds

Substance - Materials: water, steel, wood, DNA - Can be parts of other objects

FunctionalSubstance - Substances with assigned functions - Examples: fuel, medicine, building material

PhysicalArtifact - Human-made physical objects - Includes designed and repurposed objects

DesignedArtifact - Artifacts described by a Design - Most common-sense “artifacts”: cars, tools, buildings

DesignedSubstance - Substances that are also designed artifacts - Examples: pharmaceutical compounds, synthetic materials

PhysicalPlace - Locations as physical objects - Have spatial extent

SocialObject Subtree

SocialObject - Exist only through communication - Must be expressed by InformationObject - Disjoint from PhysicalObject

Description - Conceptualizations: theories, schemas, frames - Define Concepts, Roles, Tasks - Satisfied by Situations

Plan - Description of actions to be executed - Defines roles, tasks, sequencing - Examples: recipes, project plans, procedures

Project - Plan with specific goals and timeline - Typically hierarchical (sub-projects)

Workflow - Plan with explicit task sequencing - Examples: business processes, protocols

Design - Description of structure and function - Includes rationale for construction - Examples: engineering designs, architectural plans

Diagnosis - Description of system state - For control or explanation of behavior - Examples: medical diagnosis, fault analysis

Norm - Social rules and obligations - Examples: laws, ethical codes, policies

Contract - Agreement between parties (playing Party roles) - About contract object (typically a Task) - Examples: employment contracts, treaties

Goal - Description of desired Situation - Associated with Plan for achievement - Examples: objectives, aspirations

Method - Description guiding actions toward solutions - More general than Plan (alternative plans can follow same method) - Examples: scientific method, problem-solving strategies

Narrative - Description of event sequences - Interpretive framing of events - Examples: histories, stories, case studies

Theory - General conceptual framework - Examples: scientific theories, philosophical systems

Relation - Description of relation patterns - Abstract relation schemas

Pattern - Detected or proposed invariances - Examples: statistical patterns, design patterns

SocialRelation - Social relation schemas - Examples: kinship relations, institutional relations

Right - Entitlements and permissions - Legal or moral rights

Concept - Categories defined in Descriptions - Classify entities in contexts

Role - Concept for classifying agents in contexts - Examples: “student”, “driver”, “parent”

Task - EventType describing how events should be interpreted/executed - Examples: “boiling”, “reviewing”, “driving”

EventType - Concept for classifying events - Defined in Descriptions, classifies Events

LocalConcept - Concept defined in exactly one Description - Specific to single conceptual frame

Parameter - Concept for classifying Regions - Constraints or selections on values - Examples: “VeryHigh”, “Meter” (unit of measure)

UnitOfMeasure - Parameter for measurement units - Examples: meter, kilogram, second, dollar

Collection - Container for entities sharing properties - First-order entity (not a logical class) - Not an aggregate (see ObjectAggregate) - Examples: “all vintage saxophones”, “the nurses”

Collective - Collection whose members are agents - Can act “as if” agents (facon de parler) - Examples: “the nurses”, “the fans”

Configuration - Collection whose members are unified by a Description - Emerges from composed entities - Examples: parts of a book (unified by physical structure)

TypeCollection - Collection of entities of a same type - Examples: all instances of a class

Place - Socially constructed locations - Examples: countries, neighborhoods, institutions

Quality Branch

Quality - Individual aspects of entities - Dependent on bearer entity - Has Region (value in dimensional space) - Examples: specific yellowness, specific height

Situation Branch

Situation - Contextualized views satisfying Descriptions - Reified n-ary relations - Include multiple entities via `isSettingFor`

Classification - Time-indexed concept-entity classification - Special Situation for “X classifies Y at time T”

Parthood - Time-indexed part-whole relations - Special Situation for “X is part of Y at time T”

PlanExecution - Situation of executing a Plan - Links Actions to Tasks, Agents to Roles

WorkflowExecution - Situation of executing a Workflow - Temporal sequencing of tasks

Transition - Situation of change between states - Before/after state pairs

TimeIndexedRelation - Superclass for time-indexed Situations - Includes Classification, Parthood

6.3 ObjectAggregate

ObjectAggregate - Not a subclass of Entity (parallel hierarchy) - Aggregates of distributed objects from Collections - Physical distribution of collection members - Cannot be defined by equivalence axiom in OWL - Examples: stars in a constellation, car parts, encyclopedia entries

VII. Property Reference

7.1 Property Categories

DUL defines 112 object properties and 5 datatype properties. They can be organized by conceptual role:

1. **Participation & Co-occurrence**
2. **Mereological (Part-Whole)**
3. **Quality & Region**
4. **Description & Concept**
5. **Information & Expression**
6. **Situation & Context**
7. **Classification & Role**
8. **Spatial & Temporal**
9. **Precedence & Sequence**
10. **Generic Relations**

7.2 Participation Properties

hasParticipant / isParticipantIn - Domain: Event → Range: Object - Links events to participating objects - Symmetric across domain/range via inverse - Example: “The

discussion has participant John”

coparticipatesWith - Domain: Object → Range: Object - Symmetric relation between objects in same Event - Typically inferred from shared participation - Example: “Vitas and Jimmy coparticipate (in tennis match)”

7.3 Mereological Properties

hasPart / isPartOf - Domain: Entity → Range: Entity - Reflexive + transitive general parthood - Example: “The body has part the brain”

hasProperPart / isProperPartOf - Domain: Entity → Range: Entity - Asymmetric (irreflexive) + transitive strict parthood - Example: “2024 has proper part January”

hasComponent / isComponentOf - Domain: Entity → Range: Entity - Asymmetric, non-transitive structural parthood - Assumes Design structuring the system - Example: “The car has component the engine”

hasConstituent / isConstituentOf - Domain: Entity → Range: Entity - Cross-layer parthood (different ontological strata) - Example: “The person has constituent molecules”

includesWhole / includesPart - Domain: Parthood (Situation) - For time-indexed parthood in Situations - Example: “Parthood Situation includes whole (bike) and part (rack)”

7.4 Quality & Region Properties

hasQuality / isQualityOf - Domain: Entity → Range: Quality - Links entities to their individual qualities - Example: “Dmitri has quality (his skin color)”

hasRegion / isRegionFor - Domain: Entity → Range: Region - Links entities (esp. Qualities) to values - Example: “The yellowness has region Yellow₁”

hasConstraint / isConstraintFor - Domain: Entity → Range: Parameter - Generic constraints encoded as Parameters - Example: “MinimumAge is constraint for Driver role”

hasParameter / isParameterFor - Domain: Concept → Range: Parameter - Concepts have Parameters constraining classified entities - Example: “4WheelDriver Role has parameter MinimumAge”

7.5 Description & Concept Properties

defines / isDefinedIn - Domain: Description → Range: Concept - Descriptions define Concepts - Example: “Traffic Law defines Vehicle concept”

definesRole / isRoleDefinedIn - Domain: Description → Range: Role - Specialization for Role definition - Example: “Recipe defines Ingredient role”

definesTask / isTaskDefinedIn - Domain: Description → Range: Task - Specialization for Task definition - Example: “Recipe defines Boiling task”

describes / isDescribedBy - Domain: Description → Range: Entity - Description gives unity to entity parts/constituents - Example: “Design describes the car”

expands / isExpandedIn - Domain: Description → Range: Description - Partial order: one description expands another - By adding parts or refining concepts - Example: “Extended protocol expands basic protocol”

7.6 Information & Expression Properties

expresses / isExpressedBy - Domain: InformationObject → Range: SocialObject - InformationObjects express meanings (SocialObjects) - Example: “The term ‘Beehive’ expresses the Concept Beehive”

expressesConcept / isConceptExpressedBy - Domain: InformationObject → Range: Concept - Specialization for Concept expression - Example: “The word ‘dog’ expresses Concept ‘dog’”

realizes / isRealizedBy - Domain: InformationRealization → Range: InformationObject - Concrete realizations of abstract information - Example: “This book copy realizes the Constitution text”

concretelyExpresses - Domain: InformationRealization → Range: Description - Direct link from realization to expressed Description - Should be inferred via realizes + expresses chain

7.7 Situation & Context Properties

satisfies / isSatisfiedBy - Domain: Situation → Range: Description - Situations satisfy (are consistent with) Descriptions - Example: “My cooking session satisfies the recipe”

isSettingFor / hasSetting - Domain: Situation → Range: Entity - Situations are settings for included entities - Top-level property for n-ary relation projections

includesEvent / isEventIncludedIn - Domain: Situation → Range: Event - Specialization for Events in Situations

includesAction / isActionIncludedIn - Domain: Situation → Range: Action - Specialization for Actions in Situations

includesObject / isObjectIncludedIn - Domain: Situation → Range: Object - Specialization for Objects in Situations

includesAgent / isAgentIncludedIn - Domain: Situation → Range: Agent - Specialization for Agents in Situations

includesTime / isTimeIncludedIn - Domain: Situation → Range: TimeInterval - Temporal extent of Situation - Example: “Cooking session includes time (this morning)”

7.8 Classification & Role Properties

classifies / isClassifiedBy - Domain: Concept → Range: Entity (at some time) - Concepts classify entities - Often time-indexed via Classification Situation - Example: “The concept ‘Student’ classifies John”

hasRole / isRoleOf - Domain: Object → Range: Role - Objects have roles (in contexts) -

Example: “John has role Student”

hasTask / isTaskOf - Domain: Role → Range: Task - Roles have associated tasks - Example: “Student role has task ‘giving exams’”

executesTask / isExecutedIn - Domain: Action → Range: Task - Actions execute tasks - Example: “Mixing action executes Mixing task”

covers / isCoveredBy - Domain: Concept → Range: Collection - Intensional-extensional link: Concept covers Collection - Example: “Saxophone concept covers collection of saxophones”

7.9 Spatial & Temporal Properties

hasLocation / isLocationOf - Domain: Entity → Range: Entity - Generic relative spatial location - Example: “The cat has location (on the mat)”

hasTimeInterval / isTimeIntervalOf - Domain: Event → Range: TimeInterval - Temporal extent of events - Example: “The experiment has time interval August 9, 2004”

hasRegion / isRegionFor (spatial usage) - Can link Objects to SpaceRegions for absolute location - Example: “The whale has region (34°E, 20°S)”

nearTo / farFrom - Domain: Entity → Range: Entity - Symmetric distance relations - Example: “Rome is far from Beijing”

hasCommonBoundary - Domain: Entity → Range: Entity - Symmetric relation for shared boundaries - Topological, temporal, or commonsense boundaries - Example: “France has common boundary with Spain”

7.10 Precedence & Sequence Properties

precedes / follows - Domain: Entity → Range: Entity - Transitive sequence relation - Temporal or logical ordering - Example: “Year 2000 follows 1999”

directlyPrecedes / directlyFollows - Domain: Entity → Range: Entity - Intransitive (adjacent) precedence - Example: “Monday directly precedes Tuesday”

hasPrecondition / isPreconditionOf - Domain: Situation → Range: Situation - Direct precedence for Situations - Example: “Precondition of war is claiming weapons”

hasPostcondition / isPostconditionOf - Domain: Situation → Range: Situation - Direct succession for Situations - Example: “Postcondition of the plan is things settled”

7.11 Generic Relations

associatedWith - Domain: Entity → Range: Entity - Symmetric, transitive catch-all relation - Use when no more specific property applies - Example: “Italian traffic law is associated with the Italian legal system”

overlaps - Domain: Entity → Range: Entity - Symmetric overlap (mereological or spatial/temporal) - Example: “The 1990s overlap with the Cold War end”

sameSettingAs - Domain: Entity → Range: Entity - Entities in the same Situation context - Derived from shared `hasSetting` values

actsFor / isActedBy - Domain: Agent → Range: SocialAgent (esp. Organizations) - Agents act on behalf of social agents - Example: “CEO acts for the company”

actsThrough - Domain: Agent → Range: Agent - Agent acts through another agent - Example: “The board acts through the CEO”

7.12 Collection Properties

hasMember / isMemberOf - Domain: Collection → Range: Entity - Collection membership - Example: “My saxophone collection has member (this vintage alto)”

7.13 Datatype Properties

DUL includes 5 datatype properties (linking entities to literal values):

hasDataValue - Generic datatype property - Links entities to literal values (strings, numbers, etc.)

hasIntervalDate - Links TimeInterval to date literals - Typically `xsd:date` or `xsd:dateTime`

hasRegionDataValue - Links Region to literal values - For direct encoding of region values

Note: In practice, many DUL applications use datatype properties from other vocabularies (e.g., xsd, time ontology) rather than these generic ones.

VIII. Usage Guidelines

8.1 When to Use DUL

DUL is appropriate when:

1. **Domain complexity requires foundational rigor**
 - Multiple interacting domains (e.g., smart cities: urban planning + social services + IoT)
 - Need for formal consistency across domains
2. **Context-sensitivity is central**
 - Roles, classifications, or relations change over time
 - Multi-perspective modeling (same entities, different interpretations)
3. **Epistemological modeling matters**
 - Distinguishing facts from interpretations
 - Representing theories, conceptual frameworks, or competing views
4. **Social and information aspects are prominent**
 - Organizations, roles, norms, contracts
 - Documents, expressions, realizations
5. **Reusable patterns are valued**
 - Building on established ontology design patterns

- Ensuring interoperability with other DUL-based ontologies

DUL may be overkill when: - Simple taxonomies suffice (no need for reification) - Domain is purely physical/technical (no social constructs) - Time-indexing and context not needed - Lightweight vocabulary preferred (consider schema.org, Dublin Core)

8.2 How to Extend DUL

Strategy 1: Specialize Classes

Create domain-specific subclasses:

```
:MedicalProcedure rdfs:subClassOf dul:Action .
:Patient rdfs:subClassOf dul:Role .
:Hospital rdfs:subClassOf dul:Organization .
:MedicalProtocol rdfs:subClassOf dul:Plan .
```

Strategy 2: Specialize Properties

Create domain-specific sub-properties:

```
:treats rdfs:subPropertyOf dul:hasParticipant .
  # Domain: MedicalProcedure, Range: Patient

:prescribes rdfs:subPropertyOf dul:describes .
  # Domain: Prescription, Range: Medication
```

Strategy 3: Use Patterns Directly

Instantiate DUL classes without subclassing:

```
:Recipe_ChocolateCake a dul:Plan ;
  dul:defines :Role_Baker, :Task_Mixing, :Role_Ingredient .

:MyCookingSession a dul:PlanExecution ;
  dul:satisfies :Recipe_ChocolateCake ;
  dul:includesAgent :Me ;
  dul:includesAction :MixingAction_20240615 .
```

Strategy 4: Align with Other Ontologies

Map domain ontologies to DUL:

```
:MedicalProcedure owl:equivalentClass [
  owl:intersectionOf (dul:Action :DomainSpecificMedicalClass)
] .
```

```
schema:Person rdfs:subClassOf dul:NaturalPerson .
```

8.3 Common Modeling Pitfalls

Pitfall 1: Over-Reification

Problem: Using Quality-Region for every attribute

Bad:

```
:Car_123 dul:hasQuality :Color_Car123 .  
:Color_Car123 dul:hasRegion :Red .
```

Good (if individual qualities don't matter):

```
:Car_123 :hasColor :Red .
```

When to reify: Individual quality instances matter (observation tracking, temporal change of same quality)

Pitfall 2: Confusing Collections and Sets

Collection: First-order SocialObject (members share properties) **Set:** Second-order FormalEntity (mathematical set) **ObjectAggregate:** Distributed physical aggregate of collection members

Don't use Collection for mathematical sets or vice versa.

Pitfall 3: Missing Descriptions

Problem: Using Roles/Tasks without defining Descriptions

Bad:

```
:John dul:hasRole :Professor .  
# What describes the professor role? What are its constraints?
```

Good:

```
:AcademicStructure a dul:Description ;  
  dul:defines :Professor .  
  
:Professor a dul:Role ;  
  dul:isDefinedIn :AcademicStructure .  
  
:John dul:hasRole :Professor .
```

Descriptions provide the conceptual grounding for Concepts/Roles/Tasks.

Pitfall 4: Ignoring Time-Indexing

Problem: Using direct classification when time matters

Bad:

```
:Concept_Student dul:classifies :John .  
# When? Is John still a student?
```

Good:

```
:Classification_John_Student a dul:Classification ;  
  dul:satisfies :UniversityStructure ;  
  dul:isSettingFor :Concept_Student, :John, :TimeInterval_2020_2024 .
```

Use Classification Situations when temporal context is relevant.

Pitfall 5: Misusing Situations

Problem: Creating Situations without clear Descriptions or need

Bad:

```
:Situation_1 a dul:Situation ;
  dul:isSettingFor :Object_A, :Object_B .
# What description does this satisfy? What is the conceptual frame?
```

Good:

```
:Situation_1 a dul:PlanExecution ;
  dul:satisfies :Plan_XYZ ;
  dul:includesAction :Action_1 ;
  dul:includesAgent :Agent_1 .
```

Always link Situations to Descriptions that provide interpretation.

8.4 Pattern Selection Guide

Modeling Need	Pattern	Key Classes	Key Properties
Role assignment	D&S + Classification	Description, Role, Agent, Classification	defines, classifies, hasRole
Task execution	D&S + Participation	Plan, Task, Action, Agent	defines, executesTask, hasParticipant
Time-varying attributes	Quality- Region	Quality, Region, TimeInterval	hasQuality, hasRegion
Document modeling	Information Realization	InformationObject, InformationRealization	realizes, expresses
Part-whole decomposition	Mereological	Entity	hasPart, hasComponent, hasConstituent
Event participation	Participation	Event, Object, Agent	hasParticipant, coparticipatesWith
Workflow modeling	D&S + Sequence	Workflow, Task, Action, WorkflowExecution	defines, precedes, executesTask
Multi-perspective views	D&S	Description, Situation, Entity	satisfies, isSettingFor
Location modeling	Place	PhysicalPlace, Place, SpaceRegion	hasLocation, hasRegion
Organizational structure	SocialAgent + D&S	Organization, Role, Agent, Description	actsFor, hasRole, defines

8.5 Modeling Workflow

1. Identify Core Entities

- What are the main objects, agents, events in your domain?
- Classify as PhysicalObject, SocialObject, Event, etc.

2. Identify Conceptual Frames

- What theories, standards, or schemas organize your domain?
- Model as Descriptions (Plan, Norm, Design, Theory, etc.)

3. Define Concepts

- What roles, tasks, categories are defined by those frames?
- Model as Concepts (Role, Task, EventType, Parameter)
- Link via defines to Descriptions

4. Model Contexts

- What situations or contexts instantiate those frames?
- Model as Situations (PlanExecution, Classification, etc.)
- Link via satisfies to Descriptions
- Include entities via isSettingFor and specializations

5. Add Participation

- How do objects participate in events?
- Use hasParticipant and specializations

6. Add Mereology

- What part-whole relations exist?
- Choose appropriate parthood property (hasPart, hasComponent, hasConstituent)

7. Add Temporal/Spatial Info

- When/where do events occur?
- Use hasTimeInterval, hasLocation, hasRegion (for SpaceRegion)

8. Add Information Layer (if applicable)

- What information objects express domain concepts?
- Model InformationObjects and their realizations
- Use expresses, realizes

9. Refine with Parameters and Qualities (if needed)

- Add Quality-Region pattern for fine-grained attributes
- Add Parameters to Concepts for constraints

10. Validate Against Patterns

- Check that Descriptions define Concepts
- Check that Situations satisfy Descriptions
- Check that Classifications include Concepts, Entities, and Times
- Check that Properties have appropriate domains/ranges

8.6 Tool Support

Ontology Editors: - **Protégé:** Most popular OWL editor, good DUL support - **TopBraid Composer:** Commercial, strong reasoning support - **WebProtégé:** Web-based collaborative editing

Reasoners: - **HermiT:** OWL 2 reasoner, handles DUL axioms well - **Pellet:** OWL DL reasoner - **FaCT++:** Fast classification

Validation: - Check for orphan Concepts (not defined in any Description) - Check for Situations without satisfying Descriptions - Check domain/range violations (OWL reasoner will detect)

Visualization: - **OntoGraf** (Protégé plugin): Visual class hierarchy - **VOWL:** Visual notation for ontologies - **WebVOWL:** Web-based ontology visualization

8.7 Best Practices Summary

1. **Start Simple:** Use DUL patterns incrementally, don't try to model everything at once
2. **Document Decisions:** Annotate why you chose specific DUL classes/properties
3. **Reuse Patterns:** Don't reinvent—DUL patterns are tested and widely understood
4. **Prioritize Descriptions:** They are the conceptual foundation—model them carefully

5. **Use Situations for Context:** Whenever time, perspective, or framing matters
 6. **Avoid Over-Engineering:** Not every attribute needs Quality-Region; use when beneficial
 7. **Validate Frequently:** Use reasoners to catch modeling errors early
 8. **Align with Community:** Check existing DUL-based ontologies in your domain
 9. **Maintain Consistency:** Follow naming conventions (e.g., capitalize classes, camelCase properties)
 10. **Plan for Evolution:** Ontologies change—design for extensibility
-

Conclusion

DUL (DOLCE+DnS Ultralite) provides a **comprehensive foundational framework** for ontology engineering, distinguished by:

- **Epistemological sophistication:** Acknowledging that modeling is always mediated by conceptual frames (Descriptions and Situations)
- **Social-cognitive orientation:** Rich modeling of social objects, information, agency, and intentionality
- **Pattern-based design:** Reusable content patterns for common modeling challenges
- **Practical simplification:** More accessible than full DOLCE while maintaining philosophical rigor

The DUL Mindset involves: 1. **Frame-based thinking:** Reality is always interpreted through Descriptions 2. **Context awareness:** Relations and properties often depend on Situations 3. **Reification:** Making relations, attributes, and contexts first-class entities when needed 4. **Pluralism:** Multiple valid perspectives on the same reality 5. **Social construction:** Recognizing information and communication as constitutive of social reality

By adopting these principles and applying DUL's patterns, ontology engineers can create **interoperable, maintainable, and conceptually rigorous domain models** that capture not just “what exists” but “how we understand what exists”—which is, ultimately, what knowledge representation is about.

References and Further Reading

Primary Sources

- **DUL Namespace:** <http://www.ontologydesignpatterns.org/ont/dul/DUL.owl>
- **Ontology Design Patterns Portal:** <http://ontologydesignpatterns.org/>

Foundational Papers

- Gangemi, A., & Mika, P. (2003). “Understanding the Semantic Web through Descriptions and Situations”
- Masolo, C., et al. (2003). “DOLCE: a Descriptive Ontology for Linguistic and Cognitive Engineering” (WonderWeb D18)
- Borgo, S., & Masolo, C. (2009). “Foundational Choices in DOLCE”

Related Ontologies

- **DOLCE Lite-Plus:** Parent ontology
- **DOLCE Full:** Complete formalization
- **IOLite:** Information Objects ontology
- **COS/KCO:** Computational objects ontologies

Philosophical Background

- Guarino, N. (1998). “Formal Ontology and Information Systems”
- Smith, B. (2003). “Ontology” (Blackwell Guide to the Philosophy of Computing and Information)
- Searle, J. (1995). “The Construction of Social Reality”

Pattern Catalogs

- Gangemi, A. (2005). “Ontology Design Patterns for Semantic Web Content”
- Blomqvist, E., et al. (2016). “Ontology Design Patterns: Current Trends and Future Directions”

Document Version: 1.0 **DUL Version Documented:** v3.34 **Last Updated:** 2024 **Author:**
Generated from ontology analysis **License:** This documentation follows the same licensing as
DUL ontology (check <http://www.ontologydesignpatterns.org> for details)